Reflection and Traceability Report on SFWRENG 4G06 - Capstone Design Process

Team 17, DomainX

Awurama Nyarko
Haniye Hamidizadeh
Fei Xie
Ghena Hatoum

1 Changes in Response to Feedback

The below sections are the feedback and response to the feedback throughout the course, grouped by the related documents.

1.1 SRS and HA Documentation

This section summarizes the feedback received on the SRS and HA. It is divided into their respective subsection.

1.1.1 Software Requirements Specification (SRS)

This section summarizes the feedback received and resolution for the SRS.

Table 1: SRS feedback and response traceability

Source	Feedback	Change Made	Where Addressed
TA Review#116,	Sections require a brief explanation before subsection	Updated with a brief description of what the overall section contains	Issue #140; Many sections
TA Review#116,	Cite methodology paper.	When mentioning the methodology paper, we should be citing it each time.	Issue #141; Many sections

Source	Feedback	Change Made	Where Addressed
TA Review#116,	Label each goal with identifier G*, where * is the corresponding goal number. Updated all mention of goals with a reference to them.		Issue #142; Section 1.2
TA Review#116,	Page numbering should be in numbers, only the Table of Contents should be numbered in limitations.	Updated the incorrect page numbering for pages after the Table of Contents.	Issue #143; All pages
TA Review#116,	Add Dr. Smith as a customer as well in the stakeholders section.	Added reference of Dr. Smith to the customers.	Issue #144; Section 2.2
TA Review#116,	Relate goals to all the stakeholders. Indicating which goals matter to them.	Added Relevant Goals under each stakeholders with references to the stated goals.	Issue #145; Section 2
TA Review#116,	Add accessibility and compatibility to goals, since we noted that the application should be fully functional.	Since having a functional web application is assumed, it shouldn't need to be stated as an explicit goal.	Not implemented
TA Review#116,	Work partitioning table needs to include a caption and add BUC to table of acronyms.	Implemented the feedback.	Issue #148; Section 6.3
TA Review#116,	Business Data model appearing above the section that mentions it. The section should also include a description	Implemented the feedback.	Issue #149; Section 7

Source	Feedback	Change Made	Where Addressed
TA Review#116,	In the Product Use Case table, the mentioned goals should be referenced. Additionally the table also needs a caption.	Implemented the feedback.	Issue #150; Section 8.2
TA Review#116,	Several small formatting issues in NFR, such as quotation marks facing the wrong way.	Implemented the feedback.	Issue #151; Several sections
TA Review#116,	Duplicated NFRs in two sections. UH-LR1 and UH-LR2 of Learning Requirements is the same as UH-UP1 and UH-UP2 of Understandability and Politeness Requirements.	Removed UH-LR1, UH-UP1 and UH-UP2. UH-LR1 and UH-UP1 were removed due to not implementing in the end.	Issue #152; Section 11.3, Section 11.4
TA Review#116,	Duplicated NFRs in two sections. MS-SR1 of Supportability Requirements is the same as OE-PR1 of Productization Requirements.	Removed MS-SR1.	Issue #153; Section 14.2, Section 13.4
TA Review#116,	Overall, all references to external documents and software libraries should by hyperlinked.	Added hyperlinks to all referenced internal documents and external software libraries.	Issue #154; Issue #155; Overall document.
TA Review#116,	Table 10 - Development Phase Plan should list actual dates, not week numbers.	Implemented feedback.	Issue #156; Table 11.
TA Review#116,	All sections require a brief explanation before the subsection	Implemented feedback.	Issue #140; Whole document.

Source	Feedback	Change Made	Where Addressed
Peer Review#114,	Section 1.2.4 and FR12 claim the AHP process will "match what the user would calculate exactly." But Section 8.1 (Exclusions) states: "The system will not include functionality to check the accuracy of the AHP calculations." The system promises perfect AHP equivalence without including any mechanism to verify that equivalence - a logical contradiction that affects credibility and testing feasibility. Maybe add in a threshold for acceptance or some other solution?	Due to the AHP process itself being very common and having many pre-existing libraries, we said we didn't need to check it as we won't be implementation the comparison ourselves. But the wording is a bit weird so I will update the phrasing to include this additional context.	Not implemented.
Peer Review#115,	No rationale or fit criterion is provided for any requirements in the document.	Added rationale/fit criterion for the FRs. Rational/fit criterion for NFRs are out of scope due to time constraints.	Issue #255

Source	Feedback	Change Made	Where Addressed
Peer Review#118,	Excel import/export is listed as essential in Section 3.3 ("Excel integration is essential") but is still marked Pending in Open Issue OI-04. Since Excel serves as the baseline for the existing workflow, this leaves uncertainty about the system's compatibility with legacy data. The team could specify whether minimal import/export is required for the first release or whether live synchronization is planned for later phases.	Although Excel integration is required in the sense that we do need it for export, the open issue is more so clarifying whatever we need a live sync, on top. I will rephrase the open issue to clarify this.	Not implemented.

1.1.2 Hazard Analysis (HA)

This section summarizes the feedback received and resolution for the HA.

Table 2: SRS feedback and response traceability

Source	Feedback	Change Made	Where Addressed
TA Review#116,	Page numbering should be in numbers, only the Table of Contents should be numbered in limitations.	Updated the incorrect page numbering for pages after the Table of Contents.	Issue #136; All pages

Source	Feedback	Change Made	Where Addressed
TA Review#116,	Inconsistency of sentence structures in the Introduction, such as order of brackets and content within brackets.	Fixed the inconsistency.	Issue #137; Section 1
TA Review#116,	Section 3 - System Boundaries and Components should have each component labelled. The figure displaying the System Components should then use the labels in it's display.	Added labels and updated the figure	Issue #138; Section 3
TA Review#116,	Instructor comments was left in the Appendix.	Deleted the comments.	Issue #139; Appendix

1.2 Design and Design Documentation

This section summarizes the feedback received on the Design Documents, Module Guide and Module Interface Specification. It is divided into their respective subsection.

1.2.1 Module Guide (MG)

The MG revisions focused on improving consistency, abstraction, traceability, and clarity of interfaces. Feedback items were grouped by source and mapped to the final document so that each change can be traced back to the review that motivated it.

Table 3: MIS feedback and response traceability

Source	Feedback	Change Made	Where Addressed
TA Review - Rev -1#200,	Update Anticipated Changes (AC) AC7 - The visualization libraries used to display data, to include overall visualization and UI, instead of just the visualization libraries for displaying data.	Added a new AC (AC9 - The user interface features, design and colours might be changed). To encapsulate the UI elements since AC7 is specific to the graphing/plotting external libraries.	Issue #258; Section 4.1, Table 4
TA Review - Rev -1 #200,	Verify hyper references in the document is being displayed properly, some appear as ???.	Updated the refs used within the document to align with the labels used.	Issue #259; Section 6
TA Review - Rev -1#200,	Browser module needs clarification on what is supported. Are all of them supported? Such as Chrome, Firefox, Edge, Safari, etc.	Explicitly stated the supported web browser's (i.e. Chrome, Firefox, Safari, Edge)	Issue #260; Section 7.1.1
TA Review - Rev 0#200,	Delete Section 11, Design of Communication Protocols since it's not used.	Removed the section	Issue #479; Section 11
TA Review - Rev 0#200,	Remove caption of revision history table	Removed caption for the revision history table	Issue #480; Revision History Table

Source	Feedback	Change Made	Where Addressed
TA Review - Rev 0#200,	Clarify nonfunctional requirements (NFR) that are listed as Not relevant in the Trace between NFR's and Modules Table	Explicitly stated that certain NFR's (Productization (OE-PR*), Release (OE-RR*), Maintenance (MS-MR*), Supportability (MS-SR*), Adaptability (MS-AR*), Legal (CR-LR*)) are not considered, since they are out-of-scope for this document and removed them from the trace table.	Issue #478; Section 8, Table 3
Peer Review #218,	Ensure that you have identified each of your modules as the correct type in your Module Guide and Module Interface Specification. For example, I noticed that in you module guide, it states that the Automated Metrics Module is of type library. However, in your MIS, it shows that this module has state variables which library modules do not have. The same goes for library classified modules such as the Repository API Module, and Database Persistence Module.	Updated MG module's to align with the MIS.	Issue #307; Section 7

Source	Feedback	Change Made	Where Addressed
Peer Review #238,	The description of MG services is abstract, it is hard to understand the distinction between module services. Adding examples would be useful.	Added more details to the services section for the modules.	Issue #307; Section 7
Peer Review #203,	Centralization of M10 - System API Gateway Module, concerns for bottleneck	M10 is responsible for the overall flow and coordination of the information, but the other modules are responsible for fulfilling the information, so I don't think it is a bottle neck. Plus centralizing the information flow will reduce duplicating code and provide a centralized place to update any core business logic if required down the line. Additionally, it is implemented by a reliable external library (Django) that is widely used.	Not implemented
Peer Review #211,	Secrets and Services Not Clearly Differentiated	Duplicate of Peer Review #238, resolved by adding more description services.	Issue #307; Section 7
Peer Review #240,	Authentication Module Recommendation	Added M5 - User Authentication Module	Issue #307; Section 7
Peer Review #216,	Missing Connections Between Requirements and Design	The connection is already listed in Table 2 - Trace Between Functional Requirements and Module. However added an additional paragraph explaining how modules and requirements related, with an example.	Not implemented

Source	Feedback	Change Made	Where Addressed
Peer Review #224,	User Interface (UI) Design using Figma suggestion.	Not implemented since the team has already sketched out our potential UI design but will consider Figma in the future.	Not implemented
Peer Review #204,	Missing Strong Data Validation Strategy, concerns with Integrity Issues. Preference for centralized Validation Service.	Since the data is different for the different modules, I would disagree that it should be decentralized. Or else the modules would be too tightly coupled	Not implemented

1.2.2 Module Interface Specification (MIS)

The MIS revisions focused on improving consistency, abstraction, traceability, and clarity of interfaces. Feedback items were grouped by source and mapped to the final document so that each change can be traced back to the review that motivated it.

Table 4: MIS feedback and response traceability

Source	Feedback	Change Made	Where Addressed
TA Review #200, annotated MIS.pdf	Use `bool` instead of Boolean for consistency.	Revised the type notation to use `bool` consistently in the MIS. This aligned the type names with the notation table and removed inconsistent wording.	Issue #265; Section 4.3, Table 2
TA Review #200, annotated MIS.pdf	Define variables using the names intended for the implementation.	Revised variable names to better match the intended system terminology and to improve consistency between the MIS and implementation-facing documentation.	Issue #264; Sections 4.4 and 6

Source	Feedback	Change Made	Where Addressed
TA Review #200, annotated MIS.pdf	Clarify local functions, or reconfigure modules so that the module interfaces remain meaningful.	Added local functions where appropriate and revised module descriptions to better justify the structure of the affected modules.	Issue #263; Section 6
TA Review #200, annotated MIS.pdf	Clarify whether the Browser Module functions as the system's main runtime entry.	Revised the Browser Module description to state that it provides the runtime container for the client application and clarified its role in the system.	Issue #262; Section 6.1
TA Review #200, annotated MIS.pdf	Refer to the module decomposition table explicitly by number.	Revised the introductory text for module decomposition so that it explicitly refers to Table 3.	Issue #261; Section 5, Table 3
Peer Review #239	The MIS is missing abbreviations and acronyms needed for readability.	Expanded the symbols, abbreviations, and acronyms section to improve clarity and make the document easier to read independently.	Section 2, Table 1
Peer Reviews #217 and #218	Module subsections, module types, and input/output types are unclear.	Standardized the MIS structure across modules and clarified exported access programs by making inputs, outputs, and exceptions more explicit.	Section 4.3, Table 2; Section 6
Peer Reviews #212 and #223	The level of abstraction and module hierarchy need to be more precise.	Revised the module decomposition and updated module descriptions and uses clauses so that the hierarchy and abstraction boundaries are more consistent.	Section 5, Table 3; Section 6

Final Documentation Issue #501 (sub-issues #481 to #485)	Remove unnecessary text, clarify parameters and outputs, fix formatting, and hyperlink module references.	Removed unnecessary content, clarified parameter meanings and data types, defined outputs for exported access programs, corrected formatting issues, and added hyperlinks to module references.	Section 6

1.3 VnV Plan and Report

This section summarizes the feedback received on the Verification and Validation (V&V) Plan and V&V Report. The updates focused on improving traceability, clarifying verification approaches, strengthening test descriptions, and refining the structure and readability of the documents.

Table 5: V&V Plan and Report feedback and response traceability

Source	Feedback	Change Made	Where Addressed
TA Review #130, annotated VnVPlan.pdf	Traceability table contains unnecessary columns.	Simplified the traceability table by removing redundant columns and retaining only essential mappings between test IDs and requirement identifiers.	Section 4.3, Table 9
TA Review #130, annotated VnVPlan.pdf	Traceability between test cases and requirements is unclear.	Revised the traceability table to explicitly map each test ID to corresponding SRS and non-functional requirement identifiers, improving clarity and coverage justification.	Section 4.3
TA Review #130, annotated VnVPlan.pdf	Verification approach is ambiguous and difficult to follow.	Rewrote verification approach descriptions to clearly connect testing methods (unit, UI, manual, automated) with the corresponding test cases and system components.	Section 4.1; Section 4.2

Source	Feedback	Change Made	Where Addressed
TA Review #130, annotated VnVPlan.pdf	Test case descriptions are incomplete and include unnecessary columns.	Refined test case tables by removing unnecessary columns and ensuring each test clearly specifies procedure, expected outcome, and linked requirements.	Section 4.1.1–4.1.3
TA Review #130, annotated VnVPlan.pdf	Unclear whether tests are manual or automated.	Explicitly distinguished between automated and manual tests in the verification approach sections and tool descriptions.	Section 3.5; Section 3.6; Section 4
TA Review #130, annotated VnVPlan.pdf	Descriptions of testing tools are vague.	Improved Table 5 by clarifying the purpose of each tool (testing, automation, CI/CD, static analysis, and API validation).	Section 3.6, Table 5
TA Review #130, annotated VnVPlan.pdf	Unclear ownership of verification activities.	Clarified team responsibilities and roles in verification activities, including testing, reviews, and traceability management.	Section 3.1, Table 4
TA Review #130, annotated VnVPlan.pdf	Usability evaluation is not fully specified.	Expanded usability testing to include task-based evaluation, performance metrics (time, clicks), and survey-based feedback with quantitative thresholds.	Section 4.2.3; Appendix 7.2
TA Review #130, annotated VnVPlan.pdf	Ambiguity between SRS walkthroughs and code walkthroughs.	Clarified the distinction between SRS verification walkthroughs and code walkthroughs, and placed them in appropriate sections of the plan.	Section 3.2; Section 3.5
TA Review #130, annotated VnVPlan.pdf	Missing list of tables and figures based on rubric.	Added structured contents including tables and appendix references to align with documentation expectations.	Contents page

Source	Feedback	Change Made	Where Addressed
Peer Review #461	Traceability between VnV Plan and VnV Report is insufficient.	Aligned identifiers and requirement mappings across both documents so that planned tests correspond directly to reported results, improving end-to-end traceability.	VnV Plan Section 4.3; VnV Report Sections 3–4
Issue #528	Update tests for implemented features that were previously marked as in progress in earlier versions of the report.	Revised the functional requirements evaluation to reflect completed implementations rather than listing them under features in progress. Functional test cases were updated to align with the current system state.	VnV Report, Section 3
TA Review (annotated VnV Report PDF)	Some nonfunctional results are marked as *Partial*.	Expanded the corresponding nonfunctional sections to clarify the meaning of partial results and to document limitations and next steps for incomplete verification.	VnV Report, Section 4
TA Review (annotated VnV Report PDF)	Computational accuracy test case summary appears incomplete.	Completed the computational accuracy subsection by adding a full test case summary, including method, results, and analysis of AHP outputs.	VnV Report, Section 4.4

2 Extras

Our extra is a research paper on the State of the Practice for Open-Source Neural Network Software. In this research paper, we identified a shortlist of 28 open-source Neural Network (NN) software packages and measured them across 9 qualities to have a pair-wise comparison using the Analytical Hierarchy Process (AHP).
The 28 libraries are:

- Apache MXNet
- Caffe
- Captum
- CatBoost
- Chainer
- CNTK
- cuDF
- DeepLearning4J
- Gensim
- Hugging Face Transformers
- JAX
- Keras
- LightGBM
- MLflow
- NeuroLab
- NLTK
- OpenCV
- Optuna
- PaddlePaddle
- PyTensor
- PyTorch
- PyTorch Geometric
- scikit-learn
- SHAP
- TensorBoard
- TensorFlow
- TorchVision
- XGBoost

We followed the steps outline in the Methodology for Assessing the State of the Practice for Domain X paper. Our addition to the methods outline in the paper was using our tool to accomplish the data collection and comparison portions.

3 Design Iteration (LO11 (PrototypeIterate))

The design of DomainX evolved through multiple iterations based on stakeholder feedback and usability observations. The goal of these iterations was to improve usability, ensure methodological correctness, and enhance data consistency.

3.1 AHP Framework Redesign

Previous Design

In the initial version, category importance was assigned using independent sliders. Each category was given a percentage weight directly by the user.

- There was no direct comparison between categories
- Users could assign inconsistent weights
- The approach did not correctly follow the actual AHP methodology

Figure 1: Initial weight assignment using independent sliders

Final Design

We replaced this with a pairwise comparison matrix based on the AHP method. Instead of assigning percentages directly, users compare categories relative to each other using the Saaty scale.

- Weights are computed automatically
- A consistency ratio (CR) is calculated to detect unreliable inputs

This change was important because the slider approach looked simple but often produced unreliable results. The matrix-based approach is slightly more structured, but it ensures the final weights are meaningful and consistent.

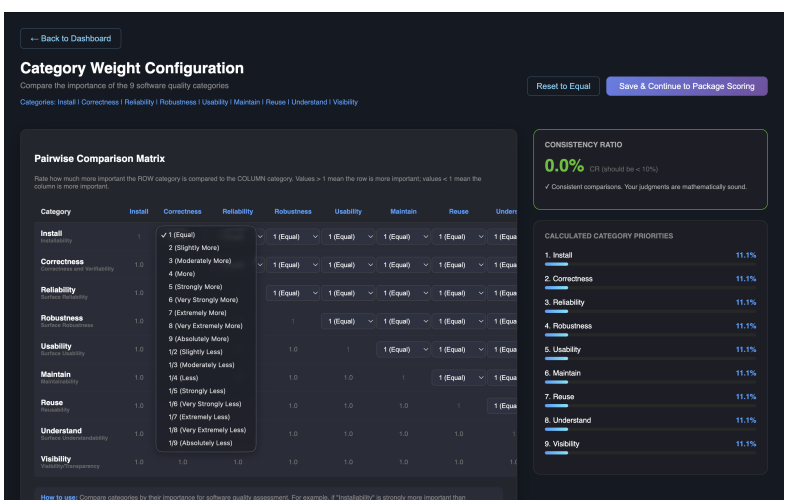

Figure 2: Pairwise comparison matrix with consistency validation

3.2 Visualization Improvements

Previous Design

The earlier version used 3D-style bar charts and only displayed a small subset of libraries.

- The 3D view made values harder to compare
- Not all libraries were visible at once

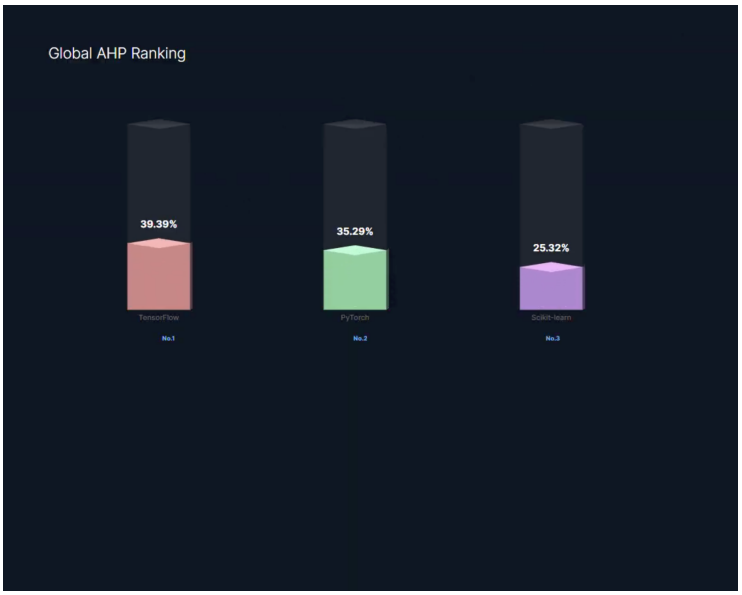

Figure 3: Previous 3D visualization with limited libraries

Final Design

We switched to a full ranking view using a standard 2D bar chart that shows all libraries.

- Easier to compare values across libraries
- No hidden data (All libraries are visible)

This change mainly came from usability issues. The previous chart looked visually appealing, but it was not practical when the number of libraries increased.

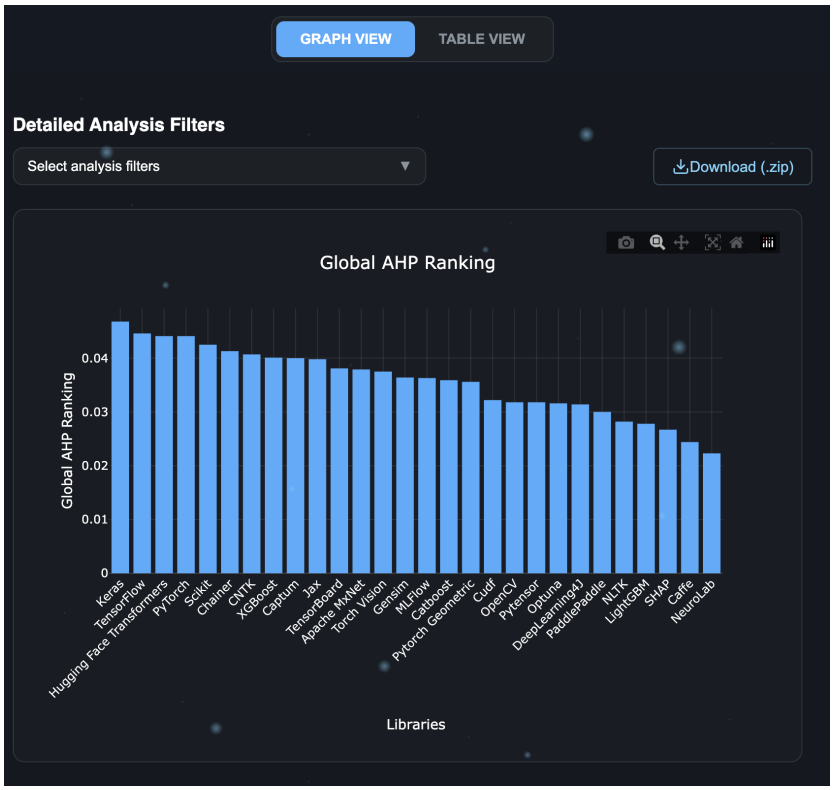

Figure 4: Improved visualization showing full ranking of libraries

3.3 Metric Definition Improvements

Previous Design

Originally, each metric had a single name field that was used both for display and internally in the system.

- Renaming a metric could break functionality (especially for automated metrics)
- There was no clear distinction between manual and automated metrics
- Input expectations were not well defined

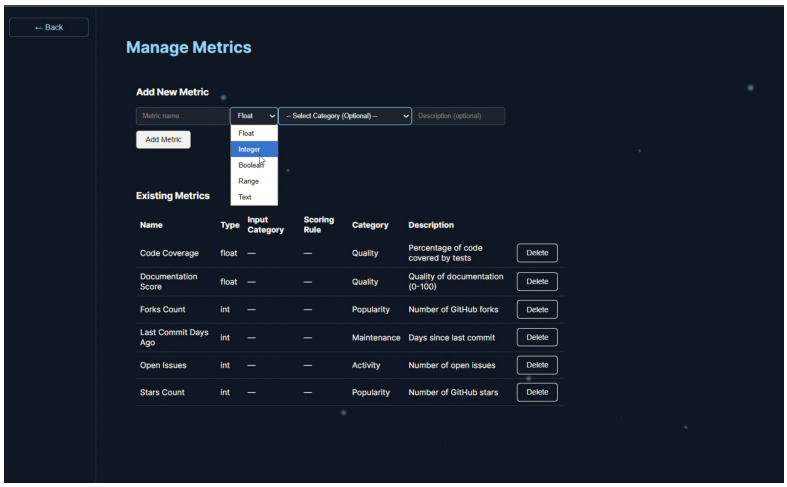

Figure 5: Metric management interface

Final Design

We redesigned the metric configuration to separate concerns more clearly:

- Introduced an internal **metric key** separate from the display name
- Allowed users to rename metrics without affecting system logic
- Added an explicit distinction between **manual** and **automated** data sources
- Supported structured metric types (Boolean, Integer, Range, etc.)
- Added input categories and scoring rules for controlled data entry

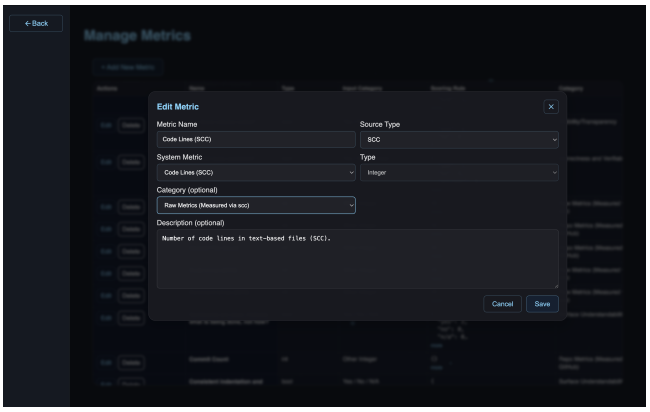

Figure 6: Configuration for automated metrics

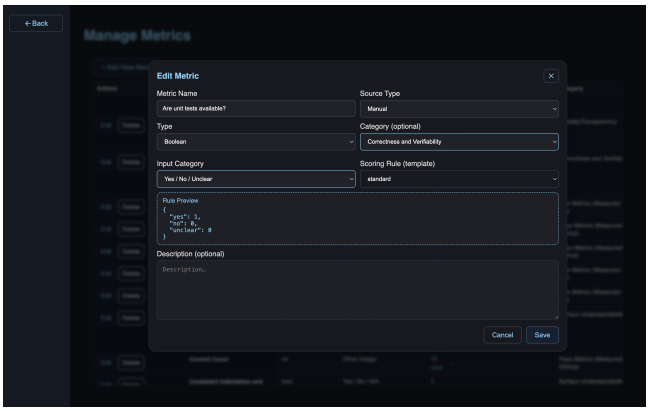

Figure 7: Configuration for manual metrics with predefined input categories

This change was mainly driven by a stakeholder requirement: they needed the ability to rename metrics freely without breaking automated data collection. Separating the internal key from the display name fixed this issue and made the system more robust.

3.4 Metric Value Input Improvements

Previous Design

Metric values were entered directly inside a table, with each row representing a library.

- Hard to navigate when many metrics were present
- Easy to lose track of which values were being edited

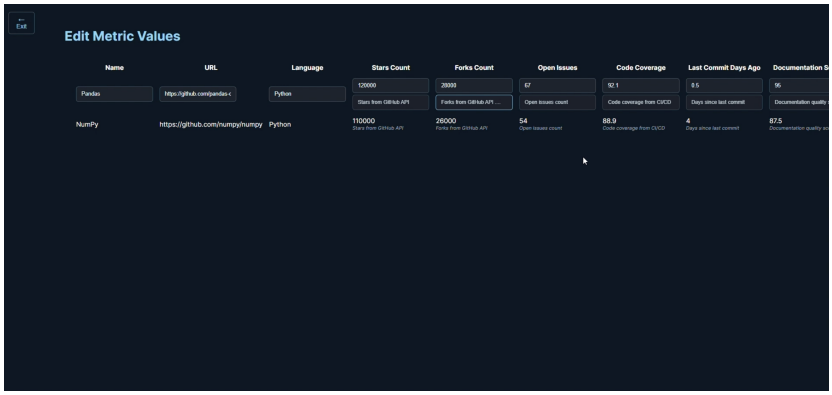

Figure 8: Row-based metric value input

Final Design
We introduced a popup-based interface for editing values.

- Users focus on one library at a time
- Inputs are grouped more clearly

This made data entry much more manageable, especially for domains with many metrics.

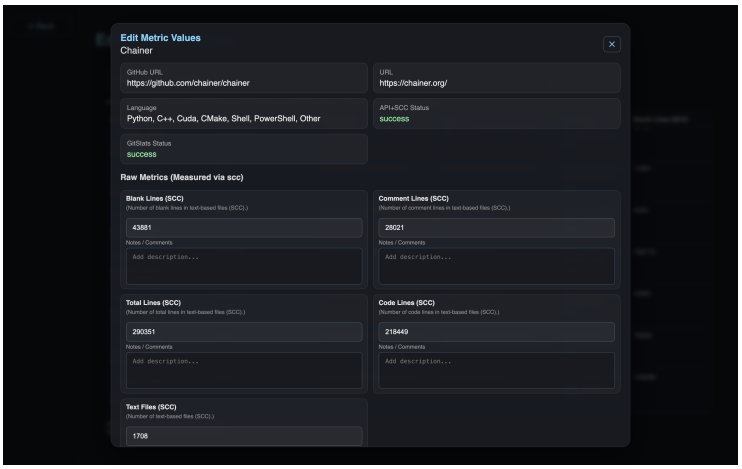

Figure 9: Popup-based metric value input interface

3.5 Data Input and Validation Improvements

Previous Design
All metric values were entered as free text.

- Flexible but inconsistent
- Same value could be entered in different formats (e.g., "yes", "Yes", "Y")

Final Design
We introduced structured inputs where possible:

- Boolean metrics now use dropdowns (Yes / No / Unclear)
- Metrics with predefined scoring (e.g., ranges or rating scales) also use dropdown selections

Free-text input is still allowed for metrics that require flexibility.
Instead of removing flexibility entirely, we limited it only where it caused problems. This helped standardize inputs without making the system too rigid.

3.6 User Feedback and Iteration

User feedback was collected through task-based testing with an external user (Ryan).

Overall Observation

Basic tasks such as domain creation, and exporting data were straightforward. However, usability issues appeared in more complex workflows involving metrics.

Key Findings and Changes

Metric Creation

Ryan noted:

> "If the scoring rule is not clicked, the system does not allow saving, but there is no clear prompt or warning message."

This showed that although required fields (e.g., metric name, scoring rule, input category) were already enforced by the system, there was no feedback indicating which fields were missing. As a result, the system appeared unresponsive when users attempted to create a metric.

Change: We added explicit validation feedback to indicate missing required fields, making it clear why a metric could not be created.

Metric Value Input

Ryan reported:

> "It was difficult to find where to fill in the manual metrics... I had to scroll or search to locate them."

This showed that the table-based input interface did not scale well.

Change: We introduced a popup-based interface to allow users to focus on one library at a time.

Input Consistency

Free-text inputs led to inconsistent data entry.

Change: We added dropdown inputs for metrics with predefined values (e.g., Boolean, ranges, scoring scales), while keeping free-text where needed.

Summary

The feedback helped identify usability gaps in metric-related workflows. The resulting changes focused on improving clarity, reducing user effort, and ensuring more consistent data entry.

4 Design Decisions (LO12)

This section reflects on the key design decisions made during the development of the DomainX tool, organized by the limitations, assumptions, and constraints that influenced them.

4.1 Limitations

The primary limitation we faced was the project timeline. Development had to be completed within a single academic year, which required us to prioritize core functionality over comprehensive features. This time constraint directly influenced several architectural decisions:

- **Modular Backend Architecture:** We chose a modular Django-based architecture rather than a monolithic design. This allowed team members to work on different modules (e.g., domain management, metric collection, AHP calculations) in parallel without blocking each other, accelerating development.
- **Feature Prioritization:** Due to time constraints, we prioritized essential features such as user authentication, domain management, metric collection, and AHP ranking over optional features like sensitivity analysis, bulk data upload, and logging changes. This ensured that the core research workflow remained functional while less critical features were deferred.
- **Automated Testing Focus:** While we aimed for 80% code coverage, time limitations prevented us from achieving this target. We focused testing efforts on critical modules (e.g., AHP calculations, data validation) where failures would have the greatest impact, while lower-risk areas received less coverage.
- **Simplified Deployment:** We opted for a traditional Django + React deployment over a more complex microservice architecture to reduce setup time and maintenance overhead, given the short development window.

4.2 Assumptions

Several assumptions guided our design decisions, based on the expected users and environment:

- **User Technical Proficiency:** We assumed that users (researchers working with Dr. Smith) have basic technical proficiency, including familiarity with web browsers, GitHub, and research methodologies. This allowed us to design a web-based interface without extensive onboarding tutorials, though we still provided tooltips and inline help for metric input fields.
- **Stable Internet Connection:** We assumed users would have reliable internet access, as the tool depends on external APIs (GitHub, PyPI) for automated data collection. This assumption influenced our decision not to implement offline caching mechanisms.
- **GitHub API Availability:** We assumed that GitHub APIs would remain accessible and within rate limits during typical usage. We implemented basic error handling but did not build complex retry or fallback mechanisms due to time constraints.

- **Researcher Collaboration:** We assumed that multiple researchers would collaborate on the same domains, requiring role-based access control. This led us to implement Contributor and Viewer roles to manage edit permissions.

4.3 Constraints

Various external and internal constraints shaped the final design of the system:

- **Time Constraint (Schedule):** The most significant constraint was the fixed capstone timeline, with a hard deadline for the final release. This forced us to make deliberate trade-offs:
  - We prioritized security features (password encryption, role-based access) over sensitivity testing, as data protection was deemed more critical for user trust.
  - We implemented a working AHP calculation matching the LBM paper methodology but deferred comprehensive sensitivity analysis to future work.
  - We focused on stability and correctness of core features rather than exhaustive performance optimization.
- **Budget Constraint:** The project could not exceed $125 in hosting costs. This constraint led us to:
  - Use McMaster University's existing infrastructure when possible, rather than paid cloud services.
  - Optimize database queries to reduce server load, allowing us to run on lower-cost instances.
  - Avoid third-party paid APIs, relying instead on free public APIs (GitHub, PyPI) with rate limits.
- **Technical Constraints:** GitHub API rate limits (60 requests per hour for unauthenticated users, 5000 for authenticated) constrained how many libraries could be analyzed in a single session. We mitigated this by using authenticated requests where possible and adding status indicators for long-running analyses.
- **Research Methodology Constraint:** The AHP calculations had to follow the exact methodology from the LBM research paper. This constrained our implementation to use the specific pairwise comparison formula:

  $$A_{jk} = \begin{cases} \min(9, x_{sub} - y_{sub} + 1) & \text{if } x_{sub} \geq y_{sub} \\ 1/\min(9, y_{sub} - x_{sub} + 1) & \text{if } x_{sub} < y_{sub} \end{cases}$$

  We could not deviate from this formula, even if alternative approaches might have been simpler to implement.

- **User Access Constraints:** The system had to support both authenticated researchers (who can edit data) and public viewers (read-only access). This led us to implement role-based access control with Django's authentication system, ensuring that unauthorized users cannot modify domain data.
- **Platform Compatibility:** The tool had to run on standard desktop browsers (Chrome, Firefox, Edge) without requiring additional software installation. This constrained us to use web technologies (HTML/CSS/JavaScript for frontend, Python/Django for backend) rather than platform-specific solutions.

4.4 Design Decisions Summary

The following table summarizes the key design decisions and their justifications:

Table 6: Summary of Design Decisions

Decision Area	**Choice Made**	**Justification**
Backend Framework	Django	Rapid development, built-in admin, ORM, and authentication
Frontend Framework	React	Component reusability, responsive UI, large ecosystem
Database	MySQL (production), SQLite (development)	MySQL for scalability, SQLite for local testing
AHP Implementation	Custom pairwise formula	Required by LBM research paper methodology
Authentication	Django built-in with role-based access	Secure, meets access control requirements
Automated Metrics	GitHub API + GitStats + SCC	Free, well-documented, meets data collection needs
Testing	pytest with GitHub Actions CI	Automated, integrates with development workflow
Deployment	McMaster University infrastructure	Cost-free, meets budget constraint
Security	Password hashing (bcrypt), role-based access	Protects user data, prevents unauthorized edits

5 Economic Considerations (LO23)

Our project is meant to be an open-source tool to be used for the research community, both active researchers and students.

Currently, we already have one user, the master's student who is researching for the Robotics software domain. The number of potential users is at minimum the amount of people who would be researching a domain.

Since our project is open source, marketing efforts would focus on raising awareness rather than selling a product. Strategies could include:

1. Promoting through word-of-mouth by Dr. Spencer Smith, development team and past users of the tool.

2. Presenting our research paper that mentions the use of this tool at academic conferences.

3. Posting online about our tool for anyone who is interested in seeing the results of this type of research.

6 Reflection on Project Management (LO24)

6.1 How Does Your Project Management Compare to Your Development Plan

Our team mostly followed the Development Plan we laid out. But as the project progressed, we made adjustments to better align with the reality of the project, and update the Development Plan accordingly.

Team Meeting Plan: The team meeting plan originally stated that we would meet as a team every Monday virtually. But starting with the second semester, we started to meet when needed. This is due to the nature of our team, where we had two subteams so oftentimes, not everyone had to attend every meeting.

Team Communication Plan: Originally, we planned on only using Microsoft Team's to communicate. But for our project, we wanted a communication platform that allowed for multiple different chat rooms, which Team's couldn't do. Therefore, we updated our communication plan and used Discord as our team's main communication platform.

Team Member Roles: The roles mostly stayed the same, the only change was the Team Leader moving from Ghena to Fei.

Workflow Plan: At the start, the workflow plan wasn't followed as tightly. Members weren't properly creating issues for their work, or labeling the issues themselves. However, towards the second half of the project, the workflow plan was followed exactly.

We followed the expected technology to build the web application, namely React for frontend, Django for backend, and MySQL for database. And implemented the linters PEP 8, Black, isort, and Flake8.

Since McMaster actually deploys its application on-premise, we did not need to use a cloud platform like Amazon Web Services (AWS).

6.2 What Went Well?

With the implementation of GitHub workflows and Run & Debug on VSCode, it was extremely easy to develop the application locally. Later on as we added testing frameworks and mandated the requirement that new features must have unit tests, and pull requests cannot be merged unless all unit test passes, it removed a lot of uncertainty that new changes would break existing systems. Furthermore, with pre-commit linting tools automating most syntax issues, the overall development was easy.

On the infrastructure side, since we used docker containers, deploying new changes was very easy, since all we need was pull the updated GitHub code and recreate the containers.

Being strict on documentation, particularly for development and infrastructure purposes reduced a lot of questions and helped get new members of the development team up and running quickly.

6.3 What Went Wrong?

Since everyone on the team was busy, there were times when team members had to wait for extended periods for their pull requests to be reviewed. Additionally, team members had different experiences and expectations regarding how issues should be structured. Some members weren't creating issues for the tasks they were working on, while others were creating too many unnecessary issues. This was detrimental because the number of issues opened and closed was a key metric used to track team contributions.

6.4 What Would You Do Differently Next Time?

Next time, we would use the GitHub Project board more frequently. While it requires some effort to set up and maintain, it greatly improves transparency by making it easier to see what other team members are working on. Since we aren't holding daily stand-up meetings, the board would help keep everyone on track and allow us to identify potential issues earlier.

7 Ethics and Equity (LO18)

We incorporated equity throughout the development process by establishing a team expectation plan that included clear communication and problem resolution guidelines. This created an environment where all team members could contribute ideas, provide feedback, and voice concerns safely and respectfully, ensuring that diverse perspectives were considered in decision-making.

In the design of the product, we applied principles of universal design by maintaining a consistent layout across all pages of the web application, making it easier for users to navigate and understand the interface. We also provided clear and timely user feedback to ensure that users are informed about system

actions and outcomes. These design choices help make the application more accessible and intuitive for users with varying levels of experience and ability.

Overall, our approach supports both equitable collaboration within the team and an inclusive user experience for all stakeholders.

8 Reflection on Capstone

Overall, our capstone project allowed us to further strengthen both our web development and project management skills. As we were responsible for the entire lifecycle of the application. From the problem statement to designing and implementing the solution, we each took on multiple roles throughout the process.

This experience closely mirrored real-world industry expectations, where teams must work within deadlines and constraints. We had to make thoughtful decisions about prioritization, recognizing that limited time and resources meant not every feature could be implemented. As a result, we focused on delivering the most impactful functionality while maintaining quality.

Through this process, we gained valuable experience in balancing technical development with planning, communication, and adaptability, all of which are essential skills in a professional software engineering environment.

8.1 Which Courses Were Relevant

SFWRENG 4HC3: Human Computer Interfaces
SFWRENG 3DB3: Databases

The above two courses were particularly relevant to our capstone, due to our project being a web application with users. SFWRENG 3DB3 was particularly helpful during the design process, when we had to design our database to accommodate our business logic.

8.2 Knowledge/Skills Outside of Courses

Web development and project management experience from past co-ops tremendously helped with the project. Since many members of the group was already familiar with certain tools, libraries and frameworks, we were able to play into our strengths and build our technology stack on what was already familiar.